

DATA 260 - Homework 3

Student Name: Priyank Mehta

SJSU ID: 019152562

GitHub Repository: <https://github.com/priyankmehta5/data260-2562>

1. Personal Configuration

This report documents the implementation and evaluation of DATA 260 Homework 3. The project extends the existing municipal transit incident application with FastAPI authentication, session management, protected routes, responsive Bootstrap templates, and a retrieval-only comparison of three LlamaIndex chunking techniques.

My SID4 is 2562, producing a DOMAIN_ID of 2. The assigned application port is 8762, calculated as $8000 + (2562 \bmod 900)$. Therefore, the assigned domain remains Municipal Transit Incidents.

Configuration value	Result
SID4	2562
PORT_BASE	8762
PREFIX	s2562
SEED	2562
VERIFY_SEED	262562
DOMAIN_ID	2
Assigned domain	Municipal transit incidents
Processor	Intel(R) Core(TM) i7-8750H CPU @ 2.20 GHz
Memory	15.86 GB RAM
Operating system	Windows 10
Dedicated GPU	NVIDIA GeForce GTX 1050 Ti
Integrated GPU	Intel(R) UHD Graphics 630
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (CPU)
Tested implementation commit	da28c955e4a11c21d367707d6735c829890e715e

2. Part 1 - FastAPI Authentication System

The authentication system is implemented as a separate FastAPI router. It provides public home and login routes, a protected dashboard, logout, signed session storage, an activity-based idle timeout, and responsive Bootstrap pages for the municipal transit domain.

1. 2.1 Home Page

```
code > api > routers > auth.py > login_page
49
50 @router.get("/")
51 def home(request: Request):
52     return templates.TemplateResponse(
53         request=request,
54         name="index.html",
55         context={
56             "user": current_user(request)
57         }
58     )
```

Figure 1. Home route rendering the domain-specific index template using the current session user.

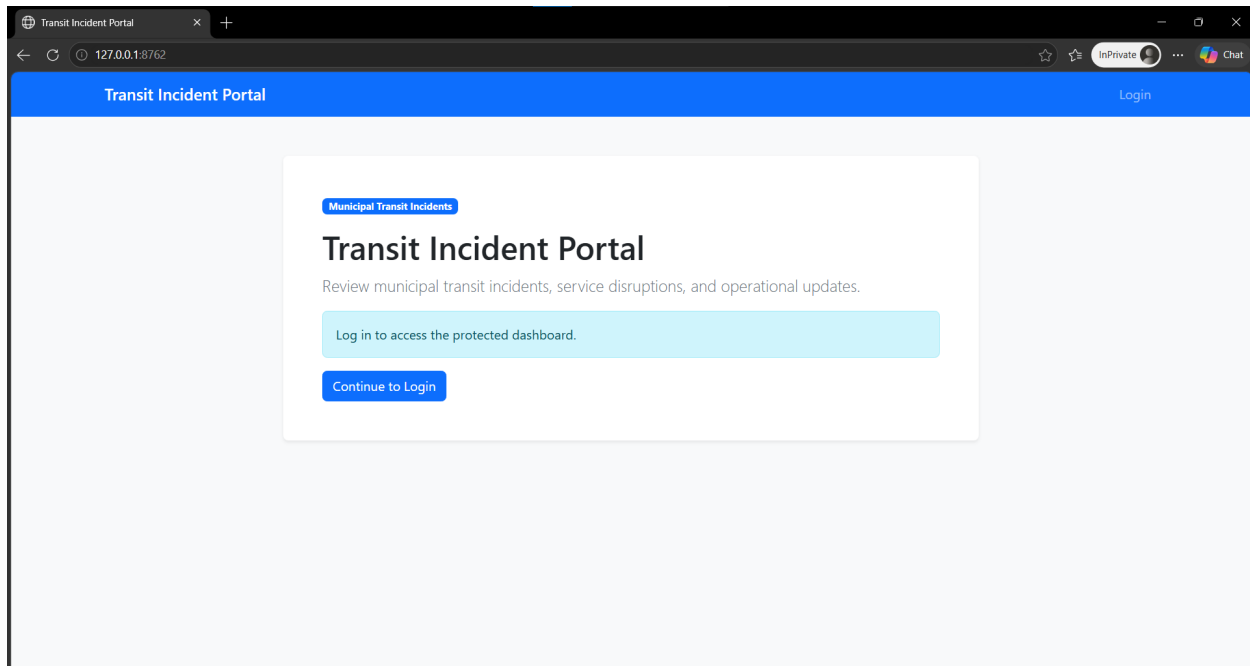


Figure 2. Municipal Transit Incident Portal home page displayed for a logged-out user.

The public home page loaded without authentication and displayed the domain-specific welcome message and login link. When a session is active, the same route can display dashboard and logout navigation.

2. 2.2 Login and Credential Validation

```
@router.get("/login")
def login_page(
    request: Request,
    expired: int = 0
):
    return templates.TemplateResponse(
        request=request,
        name="login.html",
        context={
            "user": current_user(request),
            "error": None,
            "expired": bool(expired)
        }
    )
```

Figure 3. Login-page route providing authentication status and alert values to the template.

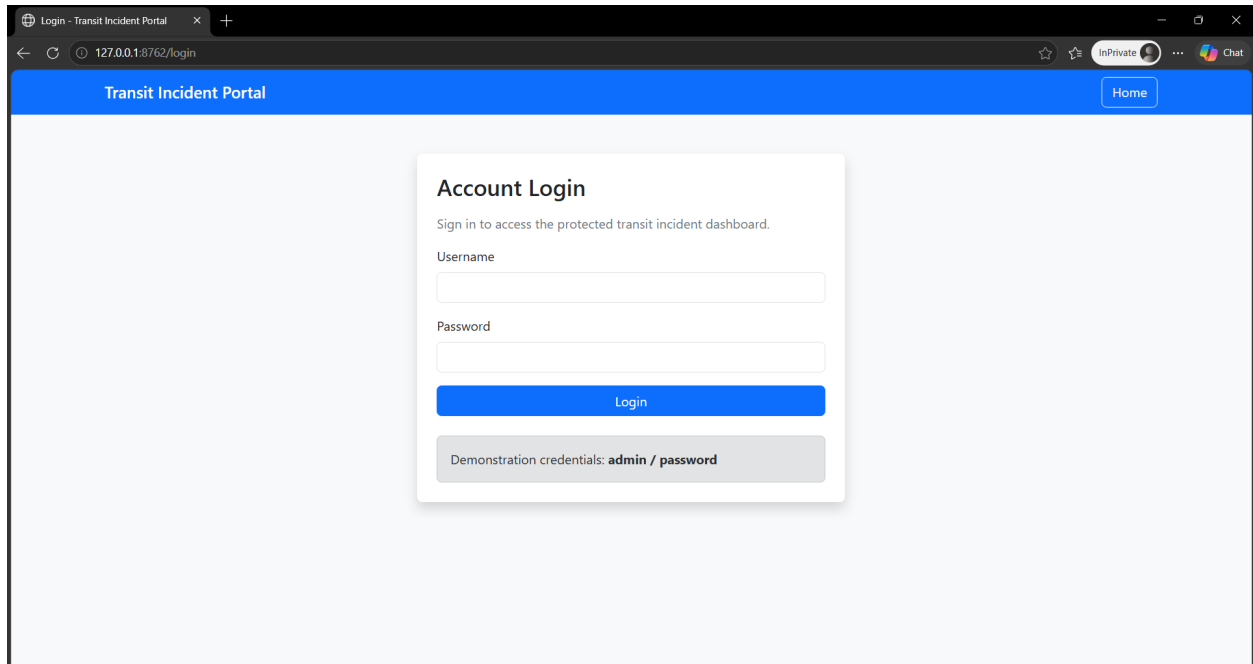


Figure 4. Bootstrap login page for the Municipal Transit Incident Portal.

```

code > api > routers > auth.py > login_page
77 @router.post("/login")
78 def login(
79     request: Request,
80     username: str = Form(...),
81     password: str = Form(...)
82 ):
83     if (
84         username == VALID_USERNAME
85         and password == VALID_PASSWORD
86     ):
87         request.session.clear()
88         request.session["user"] = username
89         request.session["last_activity"] = time.time()
90
91         return RedirectResponse(
92             url="/dashboard",
93             status_code=HTTP_302_FOUND
94         )
95
96     return templates.TemplateResponse(
97         request=request,
98         name="login.html",
99         context={
100             "user": None,
101             "error": "Invalid username or password.",
102             "expired": False
103         },
104         status_code=401
105     )

```

Figure 5. Login-processing route validating credentials and creating the authenticated session.

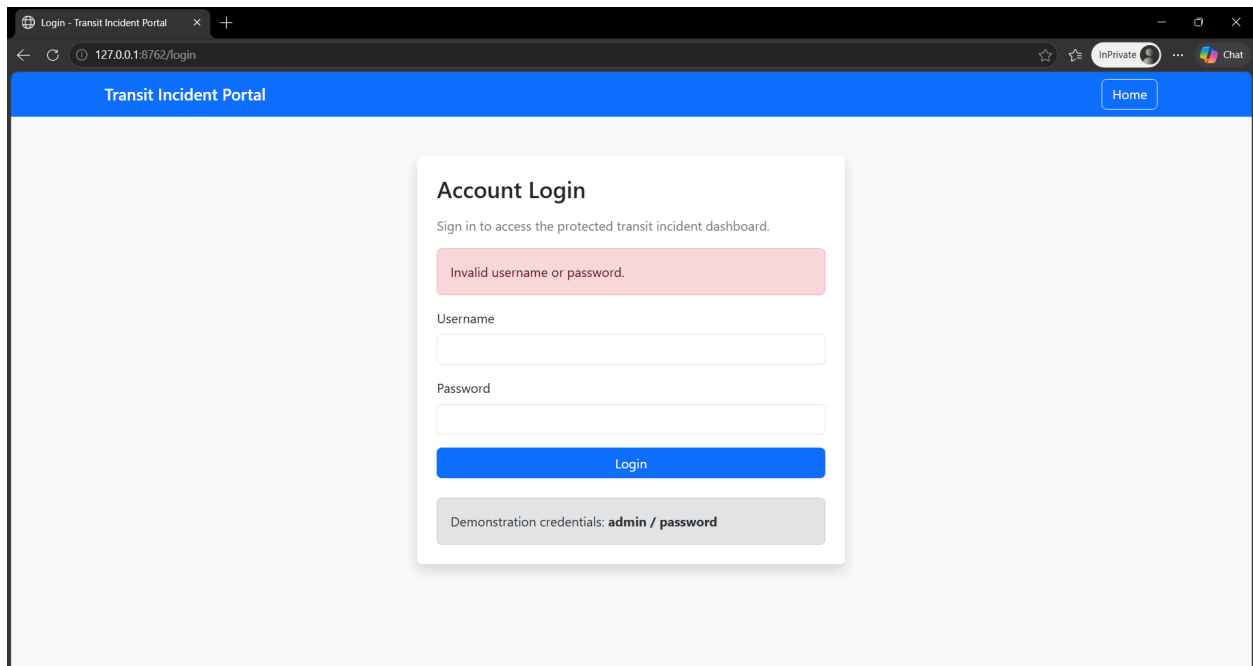


Figure 6. Bootstrap alert displayed after invalid login credentials were submitted.

The POST login route validates the submitted credentials. Valid credentials store the username and last-activity timestamp in the signed session and redirect to the dashboard. Invalid credentials create no authenticated session and display a Bootstrap alert.

2.3 Protected Dashboard

```
code > api > routers > auth.py > dashboard
107
108 @router.get("/dashboard")
109 def dashboard(request: Request):
110     if not current_user(request):
111         return RedirectResponse(
112             url="/login",
113             status_code=HTTP_302_FOUND
114         )
115
116     if session_expired(request):
117         return RedirectResponse(
118             url="/login?expired=1",
119             status_code=HTTP_302_FOUND
120         )
121
122     return templates.TemplateResponse(
123         request=request,
124         name="dashboard.html",
125         context={
126             "user": current_user(request),
127             "timeout_seconds": IDLE_TIMEOUT_SECONDS
128         }
129     )
```

Figure 7. Protected dashboard route checking authentication and idle-session expiration.

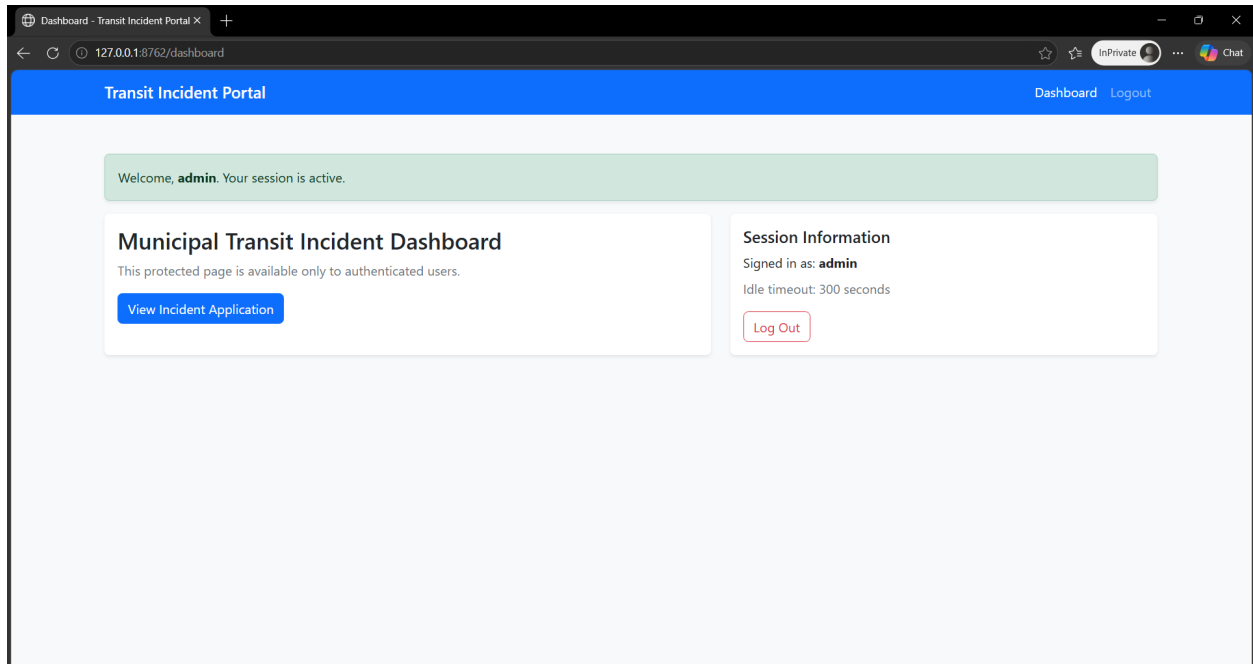


Figure 8. Protected municipal transit dashboard displayed after successful authentication.

The dashboard checks both authentication and session age before returning protected content. An unauthenticated request is redirected to the login page, while an authenticated request displays the username stored in the session.

2.4 Logout

```
code > api > routers > auth.py > dashboard
131
132 @router.get("/logout")
133 def logout(request: Request):
134     request.session.clear()
135
136     return RedirectResponse(
137         url="/",
138         status_code=HTTP_302_FOUND
139     )
```

Figure 9. Logout route clearing all session data and redirecting to the home page.

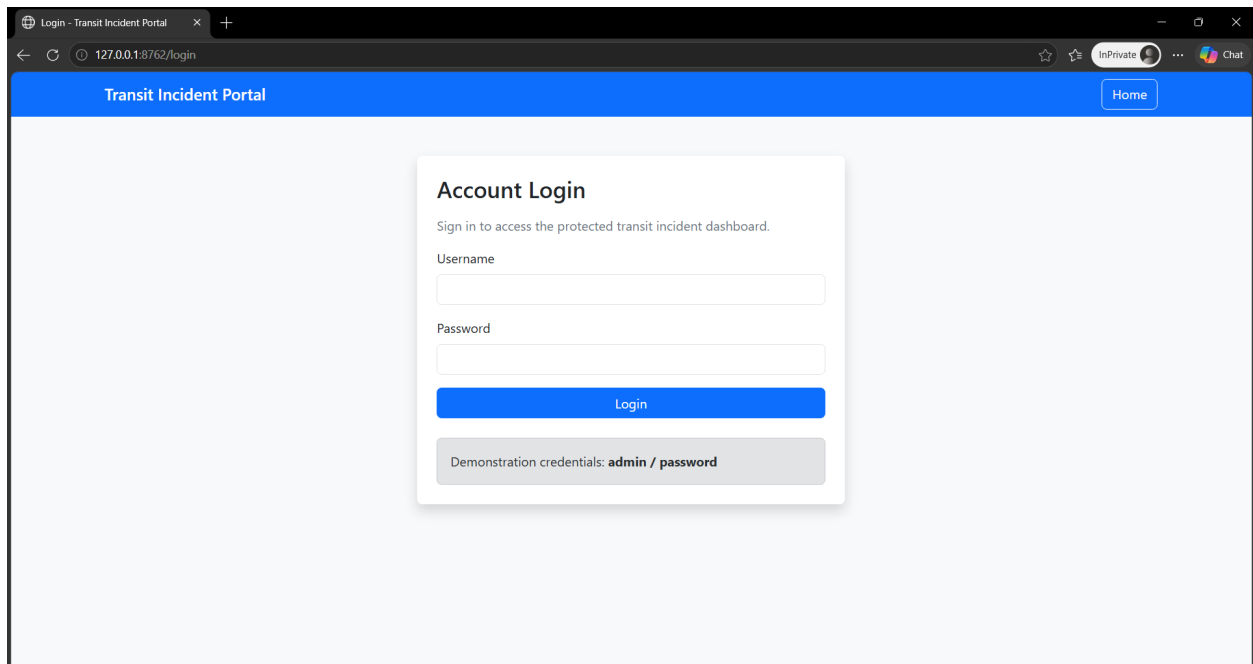


Figure 10. Attempting to reuse a logged-out session redirects the user to the login page.

Logout clears the complete session. A subsequent attempt to reuse the old session is redirected to login, confirming that redirecting alone is not being treated as logout.

2.5 Session Management and Secure Cookie

```
code > api > main.py > ...  
24 SECRET_KEY = os.getenv(  
25     "SECRET_KEY",  
26     "hw3-development-key-change-before-production"  
27 )  
28  
29 app.add_middleware(  
30     SessionMiddleware,  
31     secret_key=SECRET_KEY,  
32     session_cookie="hw3_session",  
33     max_age=1800,  
34     same_site="lax",  
35     https_only=True  
36 )
```

Figure 11. SessionMiddleware configuration enabling signed sessions and a secure session cookie.

The screenshot shows a web browser displaying the 'Municipal Transit Incident Dashboard'. The page includes a welcome message for 'admin', a 'View Incident Application' button, and session information indicating the user is signed in as 'admin' with a 300-second idle timeout. A 'Log Out' button is also present.

The Network tab is open, showing the 'login' request. The 'Set-Cookie' header is expanded, displaying the following attributes: `hw3_session=y71c2y7jogInfM1u1ivgInahc3RfYm0axZpdh101Aadlg50TUD0T12Lj04M1k2hj29_arc1fig_Rt2k5_Q8AQ5v7T25c1643T51XUj; path=/; Max-Age=1800; httponly; samesite=lax; secure`.

Figure 12. Login response Set-Cookie header showing Secure, HttpOnly, and SameSite=Lax attributes.

```
def session_expired(request: Request) -> bool:
    user = current_user(request)
    last_activity = request.session.get("last_activity")

    if not user or last_activity is None:
        return True

    idle_seconds = time.time() - float(last_activity)

    if idle_seconds > IDLE_TIMEOUT_SECONDS:
        request.session.clear()
        return True

    request.session["last_activity"] = time.time()
    return False
```

Figure 13. Activity-based idle-timeout validation using the stored last-activity timestamp.

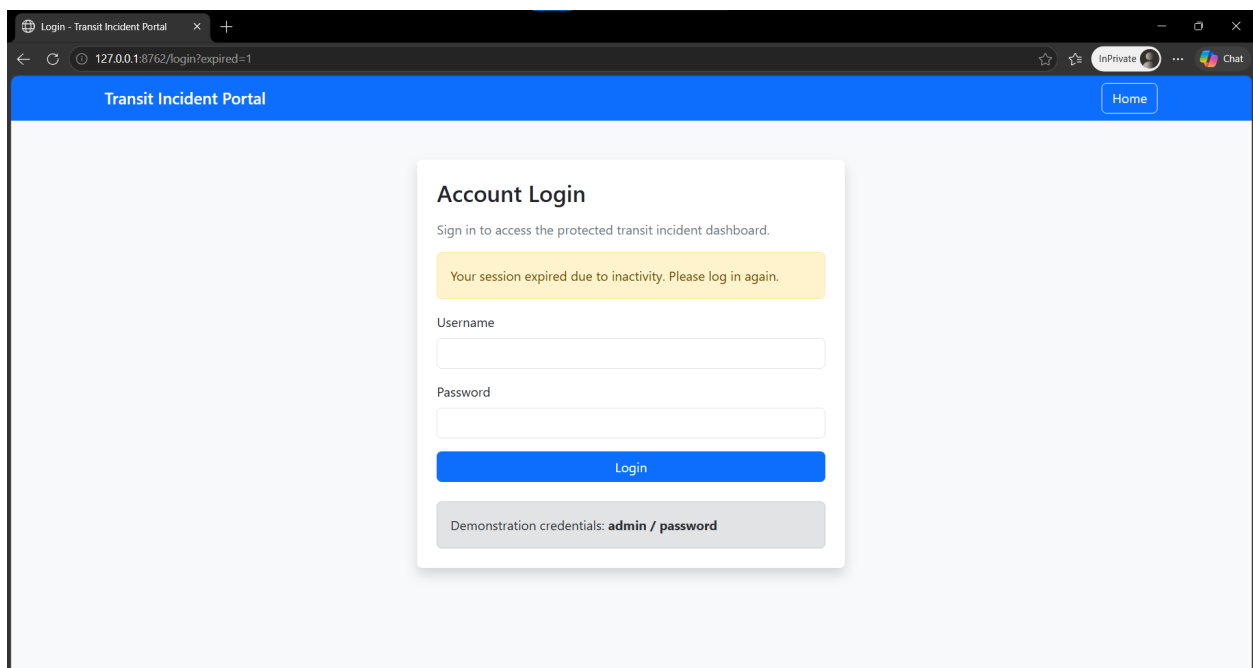


Figure 14. An idle session expires after ten seconds and cannot be reused to access the protected dashboard.

The Set-Cookie evidence shows Secure, HttpOnly, and SameSite=Lax. For the idle-timeout test, the limit was temporarily set to ten seconds. After the limit elapsed, the stored session could no longer reach /dashboard.

2.6 Bootstrap Styling and Templates

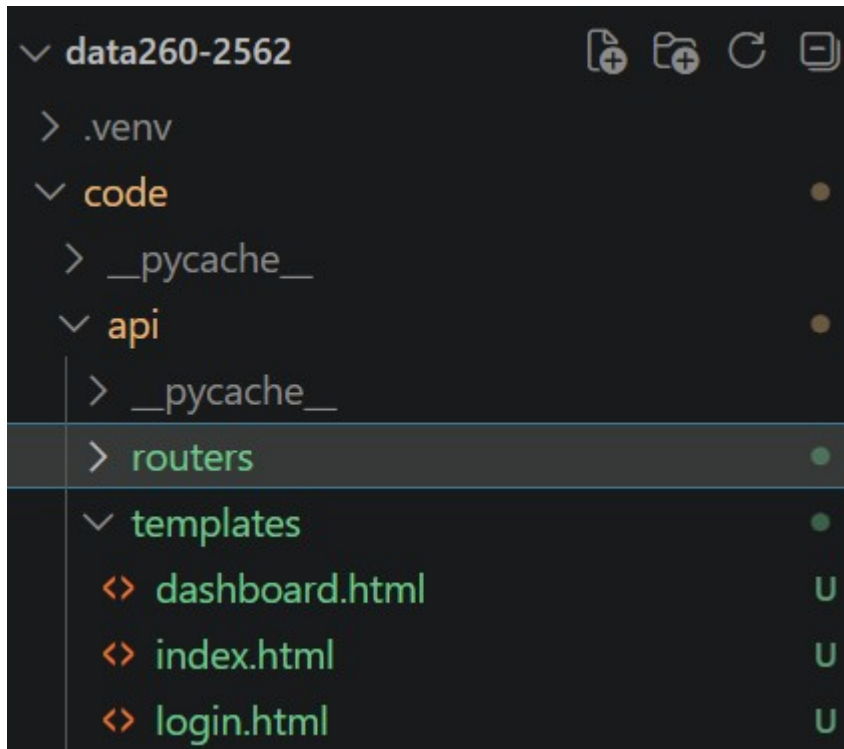


Figure 15. Templates directory containing the home, login, and dashboard pages.

The three Jinja2 templates use Bootstrap navigation bars, cards, form controls, buttons, and contextual alerts. Responsive Bootstrap containers, rows, and columns allow the pages to adapt to different screen widths. The templates also use session values supplied by auth.py to display different navigation and content for logged-in and logged-out users.

2.7 Authentication Router Code

```
code > api > routers > auth.py > ...
1  import os
2  import time
3  from pathlib import Path
4
5  from fastapi import APIRouter, Form, Request
6  from fastapi.responses import RedirectResponse
7  from fastapi.templating import Jinja2Templates
8  from starlette.status import HTTP_302_FOUND
9
10
11 router = APIRouter()
12
13 TEMPLATE_DIRECTORY = (
14 |     Path(__file__).resolve().parents[1] / "templates"
15 | )
16
17 templates = Jinja2Templates(
18 |     directory=str(TEMPLATE_DIRECTORY)
19 | )
20
21 VALID_USERNAME = "admin"
22 VALID_PASSWORD = "password"
23
24 IDLE_TIMEOUT_SECONDS = int(
25 |     os.getenv("IDLE_TIMEOUT_SECONDS", "300")
26 | )
27
28
29 def current_user(request: Request):
30 |     return request.session.get("user")
31
32
33 def session_expired(request: Request) -> bool:
34 |     user = current_user(request)
35 |     last_activity = request.session.get("last_activity")
36
37 |     if not user or last_activity is None:
38 |         return True
39
40 |     idle_seconds = time.time() - float(last_activity)
41
42 |     if idle_seconds > IDLE_TIMEOUT_SECONDS:
43 |         request.session.clear()
44 |         return True
45
46 |     request.session["last_activity"] = time.time()
47 |     return False
48
```

Figure 16. Authentication router imports, configuration, and session-timeout helper.

```
code > api > routers > auth.py > login_page
50 @router.get("/")
51 def home(request: Request):
52     return templates.TemplateResponse(
53         request=request,
54         name="index.html",
55         context={
56             "user": current_user(request)
57         }
58     )
59
60
61 @router.get("/login")
62 def login_page(
63     request: Request,
64     expired: int = 0
65 ):
66     return templates.TemplateResponse(
67         request=request,
68         name="login.html",
69         context={
70             "user": current_user(request),
71             "error": None,
72             "expired": bool(expired)
73         }
74     )
75
76
77 @router.post("/login")
78 def login(
79     request: Request,
80     username: str = Form(...),
81     password: str = Form(...)
82 ):
83     if (
84         username == VALID_USERNAME
85         and password == VALID_PASSWORD
86     ):
87         request.session.clear()
88         request.session["user"] = username
89         request.session["last_activity"] = time.time()
90
91         return RedirectResponse(
92             url="/dashboard",
93             status_code=HTTP_302_FOUND
94         )
95
96     return templates.TemplateResponse(
97         request=request,
98         name="login.html",
99         context={
100             "user": None,
101             "error": "Invalid username or password.",
102             "expired": False
103         },
104         status_code=401
```

Figure 17. Home and login routes with credential validation and session creation.

```
code > api > routers > auth.py > login_page
107
108 @router.get("/dashboard")
109 def dashboard(request: Request):
110     if not current_user(request):
111         return RedirectResponse(
112             url="/login",
113             status_code=HTTP_302_FOUND
114         )
115
116     if session_expired(request):
117         return RedirectResponse(
118             url="/login?expired=1",
119             status_code=HTTP_302_FOUND
120         )
121
122     return templates.TemplateResponse(
123         request=request,
124         name="dashboard.html",
125         context={
126             "user": current_user(request),
127             "timeout_seconds": IDLE_TIMEOUT_SECONDS
128         }
129     )
130
131
132 @router.get("/logout")
133 def logout(request: Request):
134     request.session.clear()
135
136     return RedirectResponse(
137         url="/",
138         status_code=HTTP_302_FOUND
139     )
```

Figure 18. Protected dashboard and logout routes.

Keeping the routes in `auth.py` separates authentication concerns from the existing incident API and makes the login, protection, timeout, and logout behavior easier to review.

3. Part 2 - LlamaIndex Retrieval Comparison

The graded experiment compares token-based, semantic, and sentence-window chunking over fixed local snapshots of authoritative municipal-transit documents. Each technique uses a separate local in-memory vector index and the same five precommitted questions.

```

Command Prompt - powershell
(.venv) PS D:\Grad Prep\SJSU\GitHub\data260-2562> Get-Content .\reports\hw03\CORPUS_MANIFEST.json
{
  "domain_id": 2,
  "domain": "Municipal transit incidents",
  "generated_at_utc": "2026-09-21T18:10:43.175107+00:00",
  "minimum_required_bytes": 204800,
  "total_bytes": 1933843,
  "meets_size_requirement": true,
  "files": [
    {
      "filename": "septa_norristown_collision.pdf",
      "byte_size": 1347273,
      "sha256": "66c583e43696a69acc52d188ed29cae4d198d0eaacdf473605786539c38331d3"
    },
    {
      "filename": "vta_climate_actions.pdf",
      "byte_size": 555529,
      "sha256": "8776df8adb9d1ecea03864fe662030159de67ffcd01bff90a0a08a80915cbfea"
    },
    {
      "filename": "vta_security_and_system_safety.pdf",
      "byte_size": 31041,
      "sha256": "0fab11f9aa7d1d87c9d0bcf5080dede2f35994e4c599897a109bf716ad98adda"
    }
  ]
}
(.venv) PS D:\Grad Prep\SJSU\GitHub\data260-2562>

```

Figure 19. Corpus manifest verification showing the local municipal-transit corpus size and SHA-256 file hashes.

3.1 Corpus and Source Documentation

The graded corpus contains `vta_climate_actions.pdf`, `vta_security_and_system_safety.pdf`, and `septa_norristown_collision.pdf`. `SOURCES.md` records the public source URLs and access information, while `CORPUS_MANIFEST.json` records each local filename, byte size, and SHA-256 hash. The combined corpus exceeds the required 200 KB.

```

reports > hw03 > ! questions.yaml
1  domain_id: 2
2  domain: "Municipal transit incidents"
3  top_k: 5
4
5  questions:
6    - id: "q1"
7      question: >
8        What digital communication improvements does VTA propose
9        for informing riders about emergencies and service
10       disruptions?
11     expected_answer: >
12       VTA proposes electronic signs on trains, buses, and at
13       facilities, improved SMS alerts, greater use of mobile
14       applications such as VTAlerts, and audible indicators
15       where feasible.
16     expected_source: "vta_climate_actions.pdf"
17     unique_source: true
18
19   - id: "q2"
20     question: >
21       What operational measures does VTA propose to protect
22       light rail service during extreme heat?
23     expected_answer: >
24       VTA proposes frequent track-walking inspections and
25       specific criteria for issuing rail slow or stop orders
26       to prevent or reduce overheating.
27     expected_source: "vta_climate_actions.pdf"
28     unique_source: true
29
30   - id: "q3"
31     question: >
32       Who takes primary command during a fire emergency in the
33       BART system, and with whom do they coordinate?
34     expected_answer: >
35       The local fire department assumes overall command of the
36       fire emergency scene in cooperation with BART Central
37       Control.

```

Figure 20. Predefined municipal-transit questions q1 through q3 with expected answers and source files.

```

reports > hw03 > ! questions.yaml
30     - id: "q3"
31       question: >
32         Who takes primary command during a fire emergency in the
33         BART system, and with whom do they coordinate?
34       expected_answer: >
35         The local fire department assumes overall command of the
36         fire emergency scene in cooperation with BART Central
37         Control.
38       expected_source: "vta_security_and_system_safety.pdf"
39       unique_source: true
40
41     - id: "q4"
42       question: >
43         What speed restriction and physical safety device did
44         SEPTA introduce near Norristown Station after the
45         May 2025 collision?
46       expected_answer: >
47         SEPTA reduced the maximum authorized speed from 15 mph
48         to 5 mph between signal 1N and the end of the track and
49         installed trip stops near the ends of the station tracks.
50       expected_source: "septa_norristown_collision.pdf"
51       unique_source: true
52
53     - id: "q5"
54       question: >
55         Which four possible contributing factors did the NTSB
56         exclude from the Norristown end-of-track collision?
57       expected_answer: >
58         The NTSB excluded visibility, track defects, the
59         mechanical condition of LRV 153, and drugs and alcohol.
60       expected_source: "septa_norristown_collision.pdf"
61       unique_source: true

```

Figure 21. Predefined municipal-transit questions q4 and q5 with expected answers and source files.

3.2 Precommitted Questions

The five questions cover VTA emergency communication, VTA extreme-heat rail procedures, BART fire-emergency command, SEPTA's postcollision safety actions, and factors excluded by the NTSB. Each question records an expected answer and expected source file.

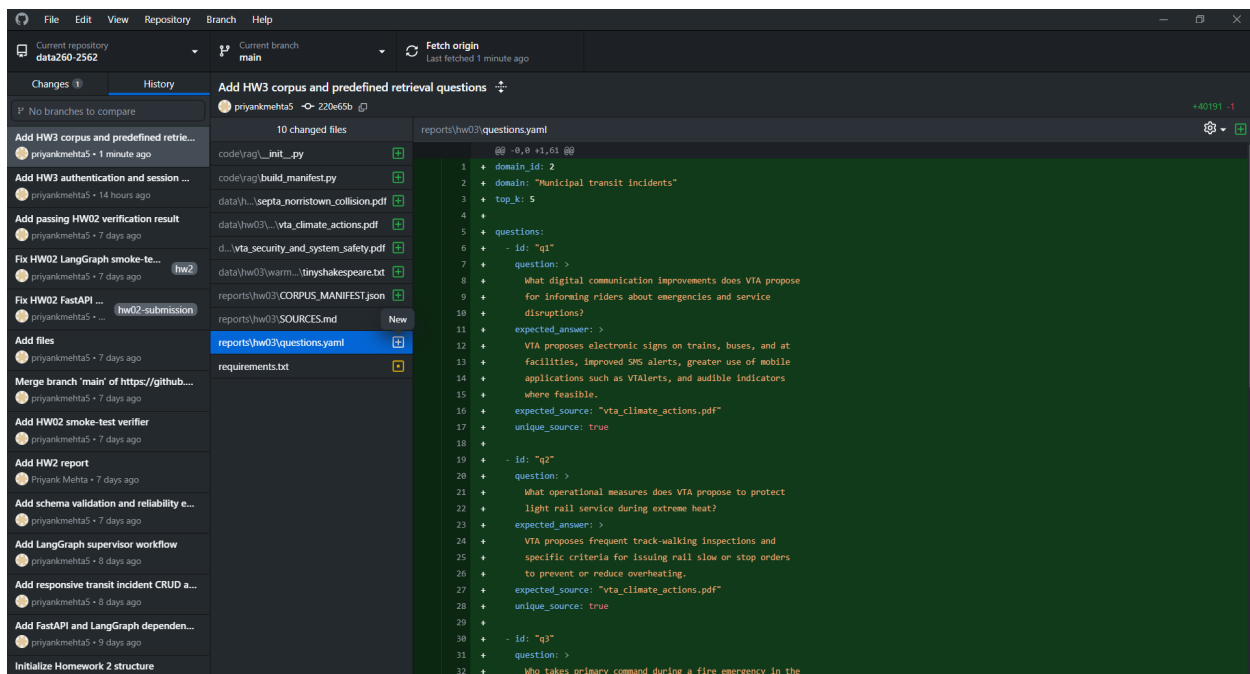


Figure 22. Git history confirming that questions.yaml was committed before the graded retrieval results were generated.

The Git history confirms that questions.yaml was committed before the graded retrieval output. This prevents expected answers or sources from being revised after observing the results.

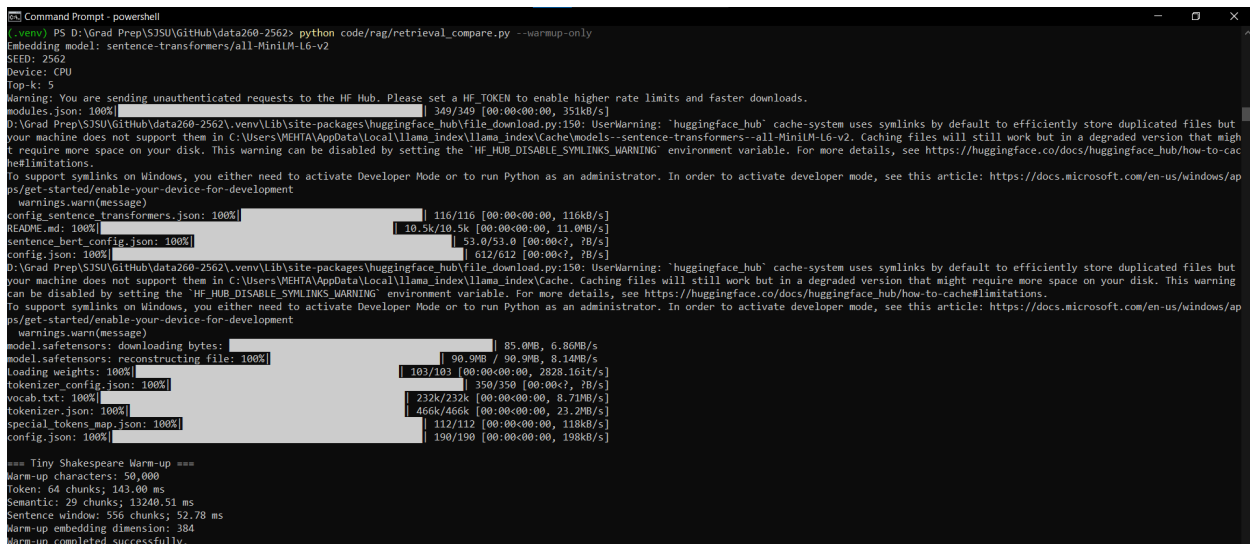


Figure 23. Successful Tiny Shakespeare warm-up using the token, semantic, and sentence-window chunking strategies.

3.3 Warm-Up

Tiny Shakespeare was used only to confirm that all three node parsers and the embedding model worked. It produced 64 token chunks, 29 semantic chunks, and 556 sentence-window chunks. It was not included in the graded retrieval measurements.

```
Command Prompt - powershell
=== Token | q1 ===
Query: What digital communication improvements does VTA propose for informing riders about emergencies and service disruptions?

Query embedding dimension: 384
Query embedding first 8 values: [0.000832, 0.04056, 0.041347, -0.069618, -0.008262, 0.024943, -0.022692, 0.021339]
Query vector shape: (384,)
Stacked document vectors shape: (5, 384)
```

Figure 24. MiniLM query embedding dimension, first eight values, and query/document vector shapes

3.4 Embeddings and Vector Shapes

The public sentence-transformers/all-MiniLM-L6-v2 model ran locally on the CPU and produced 384-dimensional vectors. Every query vector had shape (384,), and the five retrieved document vectors had stacked shape (5, 384). The first eight query values were printed for reproducibility.

```
Command Prompt - powershell
=== Sentence window | q1 ===
Query: What digital communication improvements does VTA propose for informing riders about emergencies and service disruptions?

Query embedding dimension: 384
Query embedding first 8 values: [0.000832, 0.04056, 0.041347, -0.069618, -0.008262, 0.024943, -0.022692, 0.021339]
Query vector shape: (384,)
Stacked document vectors shape: (5, 384)
Retrieval latency: 35.449 ms

Rank  Store      Cosine      Length      Preview
-----
1  0.693854  0.721303  478  Implementation Timeframe AD-CC-1.9: Improve digital infrastructure to better communicate emergency and service disruption information to riders, which may inclu...
2  0.600168  0.589870  366  External Affairs Mid-Term AD-CC-2.3: Create a web-based dashboard that underscores vulnerability assessment findings and actions that can be taken to improve re...
3  0.573993  0.529350  291  External Affairs Mid-Term AD-CC-3.1: Identify alternative transit routes and modes of transportation and develop protocols for service disruptions or temporary...
4  0.557449  0.638056  216  Conduct outreach to clarify response elements of plans and highlight VIA's capabilities to support emergency response efforts within its service area, such as p...
5  0.549885  0.581400  368  System Safety & Security Mid-Term AD-CC-3.2: Consider and fully understand all aspects of VTAs transportation network and how they will perform under evacuatio...
```

Figure 25. Sentence-window top-five retrieval results for the digital communication question, including vector-store scores, cosine similarities, chunk lengths, and previews.

3.5 Retrieval Output

For every technique and question, the output records rank, vector-store score, explicit cosine similarity, chunk length, retrieval latency, source filename, and a text preview. The sentence-window q1 result retrieved the exact VTA digital-communication action at rank 1.

```
=== Saved Machine-readable Output ===
D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw03\raw\retrieval_results.json
D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw03\raw\retrieval_results.csv
D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw03\raw\chunk_statistics.json
Retrieval rows saved: 75

Graded retrieval comparison completed.
```

Figure 26. Successful completion of the graded retrieval experiment with 75 machine-readable retrieval rows.

The complete experiment produced 75 rows: five questions times three techniques times five retrieved results. The rows were saved as machine-readable JSON and CSV together with chunk statistics.

```
(.venv) PS D:\Grad Prep\SJSU\GitHub\data260-2562> python .\code\rag\summarize_metrics.py
HW3 retrieval metrics

Token: Accuracy@1=100.0%; Recall@5=100.0%; MRR=1.000; p50=17.783 ms; p95=18.011 ms
Semantic: Accuracy@1=100.0%; Recall@5=100.0%; MRR=1.000; p50=17.435 ms; p95=19.121 ms
Sentence window: Accuracy@1=100.0%; Recall@5=100.0%; MRR=1.000; p50=35.449 ms; p95=38.264 ms

Confident false positive:
Semantic q2 retrieved vta_security_and_system_safety.pdf instead of vta_climate_actions.pdf with store score 0.5

Saved: D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw03\raw\retrieval_metrics.json
Saved: D:\Grad Prep\SJSU\GitHub\data260-2562\reports\hw03\METRICS.md
(.venv) PS D:\Grad Prep\SJSU\GitHub\data260-2562> _
```

Figure 27. Retrieval quality and latency metrics with a confident semantic-retrieval false positive for question q2.

3.6 Summary Metrics

All three techniques achieved 100% source Accuracy@1, 100% source Recall@5, and an MRR of 1.000. These source-level measurements were supplemented by manual review because a passage can come from the correct document without containing the exact answer.

```

code > rag > retrieval_compare.py > MODEL_NAME
52  MODEL_NAME = (
53      "sentence-transformers/all-MiniLM-L6-v
54  )
55
56  SEED = 2562
57  TOP_K = 5
58
59  TOKEN_CHUNK_SIZE = 256
60  TOKEN_CHUNK_OVERLAP = 32
61
62  SEMANTIC_BUFFER_SIZE = 1
63  SEMANTIC_BREAKPOINT_PERCENTILE = 95
64
65  SENTENCE_WINDOW_SIZE = 3
66
67
68  def parse_args() -> argparse.Namespace:
69      parser = argparse.ArgumentParser(
70          description=(
71              "Compare Token, Semantic, and
72              "Sentence-window retrieval."
73          )
74      )
75
76      parser.add_argument(
77          "--corpus",
78          type=Path,
79          default=DEFAULT_CORPUS,
80      )
81      parser.add_argument(

```

Figure 28. MiniLM embedding model, random seed, top-k value, and chunking configurations used in the retrieval experiment.

3.7 Fixed Configuration

The experiment used SEED 2562 and top-k 5. Token chunking used a 256-token size and 32-token overlap. Semantic chunking used buffer size 1 and breakpoint percentile 95. Sentence-window chunking used window size 3.

```
code > rag > retrieval_compare.py > create_parsers
176 def create_parsers(embed_model):
177     return {
178         "Token": TokenTextSplitter(
179             chunk_size=TOKEN_CHUNK_SIZE,
180             chunk_overlap=TOKEN_CHUNK_OVERLAP,
181         ),
182         "Semantic": (
183             SemanticSplitterNodeParser(
184                 buffer_size=SEMANTIC_BUFFER_SIZE,
185                 breakpoint_percentile_threshold=(
186                     SEMANTIC_BREAKPOINT_PERCENTILE
187                 ),
188                 embed_model=embed_model,
189             )
190         ),
191         "Sentence window": (
192             SentenceWindowNodeParser.from_defaults(
193                 window_size=SENTENCE_WINDOW_SIZE,
194                 window_metadata_key="window",
195                 original_text_metadata_key=(
196                     "original_text"
197                 ),
198             )
199         ),
200     }
```

Figure 29. Token, semantic, and sentence-window chunking techniques configured with LlamaIndex.

3.8 Three Chunking Techniques

`TokenTextSplitter` follows a fixed token budget with overlap. `SemanticSplitterNodeParser` uses embedding changes to form semantic boundaries. `SentenceWindowNodeParser` retrieves sentence-level nodes while retaining neighboring sentences in metadata.

```

code > rag > retrieval_compare.py > build_indexes
307 def build_indexes(
308     nodes_by_technique,
309     embed_model,
310 ):
311     indexes = {}
312
313     print("\n=== In-memory Indexing ===")
314
315     for technique, nodes in (
316         nodes_by_technique.items()
317     ):
318         vector_store = SimpleVectorStore()
319
320         storage_context = (
321             StorageContext.from_defaults(
322                 vector_store=vector_store
323             )
324         )
325
326         started = time.perf_counter()
327
328         indexes[technique] = VectorStoreIndex(
329             nodes,
330             storage_context=storage_context,
331             embed_model=embed_model,
332             show_progress=True,
333         )
334
335         elapsed_ms = (
336             time.perf_counter() - started
337         ) * 1000
338
339         print(
340             f"{technique}: indexed "
341             f"{len(nodes)} chunks in "
342             f"{elapsed_ms:.2f} ms"

```

Figure 30. Construction of separate local in-memory vector indexes for the three chunking techniques.

3.9 In-Memory Indexing

A separate SimpleVectorStore and VectorStoreIndex were built for each technique. The token parser produced 91 chunks averaging 1,077.01 characters; semantic parsing produced 27

chunks averaging 3,146.44 characters; sentence-window parsing produced 460 chunks averaging 184.68 characters.

```
code > rag > retrieval_compare.py > cosine_similarity
348 def cosine_similarity(
349     left: np.ndarray,
350     right: np.ndarray,
351 ) -> float:
352     denominator = (
353         np.linalg.norm(left)
354         * np.linalg.norm(right)
355     )
356
357     if denominator == 0:
358         return 0.0
359
360     return float(
361         np.dot(left, right) / denominator
362     )
363
364
365 def text_preview(
366     text: str,
367     limit: int = 160,
368 ) -> str:
369     cleaned = normalize_text(text)
370
371     if len(cleaned) <= limit:
372         return cleaned
373
374     return cleaned[:limit].rstrip() + "..."
375
376
377 def print_results_table(
378     rows: list[dict[str, Any]],
379 ) -> None:
380     header = (
381         f"{'Rank':<6}"
382         f"{'Store':<12}"
383         f"{'Cosine':<12}"
384         f"{'Length':<10}"
385         f"Preview"
386     )
387
388     print(header)
389     print("-" * 100)
```

Figure 31a. Independent cosine-similarity calculation and vector-shape inspection used to audit vector-store retrieval scores.

```
code > rag > retrieval_compare.py > cosine_similarity
377 def print_results_table(
...
391     for row in rows:
392         store_score = row["store_score"]
393
394         if store_score is None:
395             store_display = "N/A"
396         else:
397             store_display = (
398                 f"{store_score:.6f}"
399             )
400
401         print(
402             f"{row['rank']:<6}"
403             f"{store_display:<12}"
404             f"{row['cosine_similarity']:<12.6f}"
405             f"{row['chunk_length']:<10}"
406             f"{row['preview']}"
407         )
```

Figure 31b. Independent cosine-similarity calculation and vector-shape inspection used to audit vector-store retrieval scores

3.10 Independent Similarity Check

The script explicitly embeds each returned passage and calculates cosine similarity with NumPy. Both the store score and explicit cosine are retained. They can differ because the indexed sentence and the displayed window-expanded text are not always identical.

```

code > rag > retrieval_compare.py > retrieve_for_question
410 def retrieve_for_question(
464
465     row = {
466         "question_id": question_record["id"],
467         "question": query,
468         "expected_answer": (
469             question_record["expected_answer"]
470         ),
471         "expected_source": (
472             question_record["expected_source"]
473         ),
474         "unique_source": bool(
475             question_record.get(
476                 "unique_source",
477                 False,
478             )
479         ),
480         "technique": technique,
481         "rank": rank,
482         "source_file": source_file,
483         "store_score": (
484             float(item.score)
485             if item.score is not None
486             else None
487         ),
488         "cosine_similarity": (
489             cosine_similarity(
490                 query_embedding,
491                 document_embedding,
492             )
493         ),
494         "chunk_length": len(text),
495         "preview": text_preview(text),
496         "full_text": text,
497         "window_text": window_text,
498         "expected_source_match": (
499             source_file
500             == question_record[
501                 "expected_source"
502             ]
503         ),
504         "retrieval_latency_ms": latency_ms,
505     }

```

Figure 32. Machine-readable top-k retrieval record containing source, score, cosine similarity, chunk length, latency, and expected-source evaluation.

Each machine-readable result includes question ID, technique, rank, source, store score, cosine similarity, chunk length, latency, expected-source match, and preview.

```
code > rag > retrieval_compare.py > write_outputs
612 def write_outputs(
613     json_path = (
614         output_directory
615         / "retrieval_results.json"
616     )
617 )
618     json_path.write_text(
619         json.dumps(
620             structured_output,
621             indent=2,
622             ensure_ascii=False,
623         )
624         + "\n",
625         encoding="utf-8",
626     )
627
628     flat_rows = [
629         row
630         for run in runs
631         for row in run["results"]
632     ]
633
634     csv_path = (
635         output_directory
636         / "retrieval_results.csv"
637     )
638
639     fieldnames = [
640         "question_id",
641         "technique",
642         "rank",
643         "source_file",
644         "store_score",
645         "cosine_similarity",
646         "chunk_length",
647         "retrieval_latency_ms",
648         "expected_source",
649         "expected_source_match",
650         "preview",
651     ]
652
653     with csv_path.open(
654         "w",
655         encoding="utf-8",
656         newline="",
657     ) as file:
658         writer = csv.DictWriter(
659             file,
660             fieldnames=fieldnames,
661         )
662         writer.writeheader()
663
664         for row in flat_rows:
665             writer.writerow(
666                 {
667                     key: row.get(key)
668                     for key in fieldnames
669                 }
670             )
671
672     stats_path = (
673         output_directory
674         / "chunk_statistics.json"
675     )
```

Figure 33. Retrieval results and chunk statistics written to machine-readable JSON and CSV files.

retrieval_results.json and retrieval_results.csv preserve all 75 retrieval rows. chunk_statistics.json records chunk counts and lengths, and summarize_metrics.py recomputes the comparison from the saved CSV.

```
code > rag > summarize_metrics.py > calculate_metrics
79  def calculate_metrics(rows):
179      statistics.mean(reciprocal_ranks)
180      ),
181      "mean_top1_store_score": (
182          statistics.mean(
183              top_one_store_scores
184          )
185      ),
186      "mean_top1_cosine": (
187          statistics.mean(top_one_cosines)
188      ),
189      "mean_chunk_length": (
190          statistics.mean(
191              row["chunk_length"]
192              for row in technique_rows
193          )
194      ),
195      "latency_mean_ms": (
196          statistics.mean(query_latencies)
197      ),
198      "latency_p50_ms": percentile(
199          query_latencies,
200          0.50
201      ),
202      "latency_p95_ms": percentile(
203          query_latencies,
204          0.95
205      ),
206      }
207  )
208
209  false_positives.sort(
210      key=lambda item: item["store_score"],
211      reverse=True
212  )
```

Figure 34. Objective retrieval metrics and selection of the highest-scoring incorrect-source result as the confident false positive.

3.11 Confident False Positive

The highest-scoring result from an incorrect source occurred for semantic chunking on q2. The query asked about VTA extreme-heat light-rail measures, but a result from `vta_security_and_system_safety.pdf` received a store score of 0.559343 instead of coming from the expected `vta_climate_actions.pdf` source.

The embedding likely scored this passage confidently because both documents discuss VTA operations, light rail, system safety, hazards, and protective procedures. The thematic vocabulary is similar even though the retrieved passage does not contain the requested extreme-heat answer.

3.12 Retrieval Quality Comparison

The following values aggregate all five precommitted questions. Mean top-1 cosine is the average of the highest explicit cosine for each question. Mean@5 cosine is the average across all 25 results for each technique.

Technique	Chunks	Avg chunk length	Top-1 cosine	Mean@k cosine	Recall@k	Mean retrieval latency (ms)
Token	91	1,077.01	0.679	0.609	100%	16.766
Semantic	27	3,146.44	0.581	0.514	100%	17.495
Sentence window	460	184.68	0.711	0.592	100%	33.682

Table 1. Retrieval quality and efficiency comparison across the three chunking techniques.

3.13 Observations

Sentence-window chunking produced the strongest focused matches. It had the highest mean top-1 cosine, 0.711, and its leading results for q1 through q4 contained concise passages closely aligned with the questions. Sentence-level retrieval reduced unrelated material, while the stored window preserved useful context. Its mean retrieval latency was 33.682 ms, approximately twice that of the other techniques, because it indexed 460 small chunks.

Token chunking had the highest Mean@5 cosine, 0.609, and the lowest mean retrieval latency, 16.766 ms. Its overlapping chunks preserved broader context but were less focused. Semantic chunking produced only 27 very large chunks and had the lowest mean top-1 and Mean@5 cosine values. Several semantic chunks mixed relevant and unrelated content.

3.14 Conclusion

I judge sentence-window chunking to be the best technique for this corpus because it produced the highest mean top-1 cosine and the most focused answer-bearing passages. Token chunking was the best speed-oriented alternative and produced the strongest average similarity across the full top-five sets. Semantic chunking was less effective because its large chunks frequently mixed relevant and unrelated material.

4. AI Use Disclosure

4.1 What did I use an AI assistant for, and what did I do myself?

I used ChatGPT to troubleshoot installation errors, missing dependencies, PowerShell commands and CSV field-name mismatches.

I independently developed and ran the application and experiments, tested authentication and session behavior, reviewed the retrieved passages, verified the generated files, captured the required screenshots, and interpreted the final results.

4.2 What AI-produced output was wrong or unsuitable?

The AI assistant suggested downloading an official FTA PDF with a PowerShell Invoke-WebRequest command. The command was unsuitable because the FTA server denied the automated request, so the required PDF could not be downloaded.

4.3 How did I detect the problem or verify the result?

PowerShell returned an Access Denied error instead of downloading the document. I also confirmed that no valid PDF had been created in the corpus directory. This showed that the source could not be used in a reproducible automated setup.

4.4 What did I change, and why does it work now?

I replaced the inaccessible FTA document with an authoritative VTA security and system-safety PDF that was relevant to the municipal transit domain and could be downloaded successfully. I then updated SOURCES.md and regenerated CORPUS_MANIFEST.json to record the new source, file size, and SHA-256 hash. The replacement works because the corpus now contains a valid, accessible PDF from an authoritative transit agency, and the retrieval script can extract and index its text successfully.

5. Reproducibility and Evidence

Exact setup and execution commands are provided in `reports/hw03/REPRODUCIBLE.md`. Real console output and timestamps are preserved in `RUN_LOG.txt`. Machine-readable retrieval results are stored under `raw/`, and the self-check result is stored in `verification.json`.